

SOFTWARE ENGINEERING COURSE

LECTURE 2

Software Development Life cycle and Methodologies

Micheal Tuape, D.Sc

“Software Engineering: A Practitioner’s Approach” by Roger S. Pressman, published by McGraw-Hill Education, 2017.



MODULE 1 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 1	September 2 — September 6, 2024	Course Introduction	None
Week 2	September 9 — September 13, 2024	Introduction to Software Engineering	Ch. 1
Week 3	September 16 — September 20, 2024	Software Development Life Cycle & Methodologies	Ch. 2
Week 4	September 23 — September 28, 2024	Software Development Processes	Ch. 3
Week 5	October 2 — October 8, 2024	Requirements Engineering	Ch. 4
Week 6	October 9 — October 15, 2024	Software Modelling, Coding and Dev.	Ch. 5
Week 7	October 16 — October 22, 2024	Software Architectural Design	Ch. 6



LECTURE 2 AGENDA

- » Software Development Life cycle(SDLC) and methodology
- » SDLC models
 - Water Fall
 - Spiral Models
 - V model
 - Iterative
 - RAD
 - Rapid Software Development
 - Agile (XP, Scrum)





LECTURE 2 LEARNING OUTCOME

- »» Clear understanding of the Software Development Life cycle(SDLC) methodology and framework
- »» Compare and contrast the different SDLC models
- »» Develop scenarios of choosing the appropriate Model

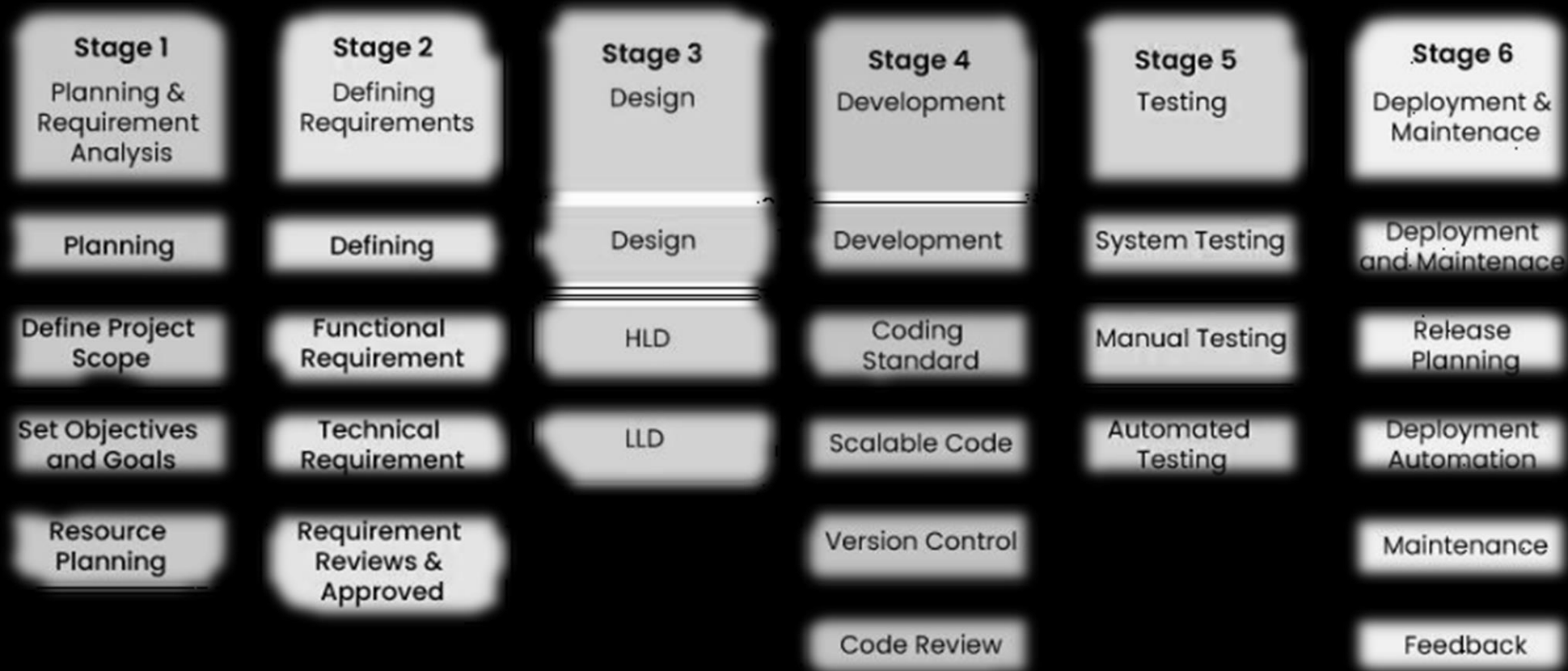


SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

- SDLC a brainchild of Winston N Boyce, is defined as a structured process that is used to design, develop, and test good-quality software.
- This methodology defines the entire procedure of software development step-by-step.
- The goal of the SDLC life cycle model is to deliver high-quality, maintainable software that meets the user's requirements.
- The SDLC outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets users' requirements.

STAGES OF THE SDLC

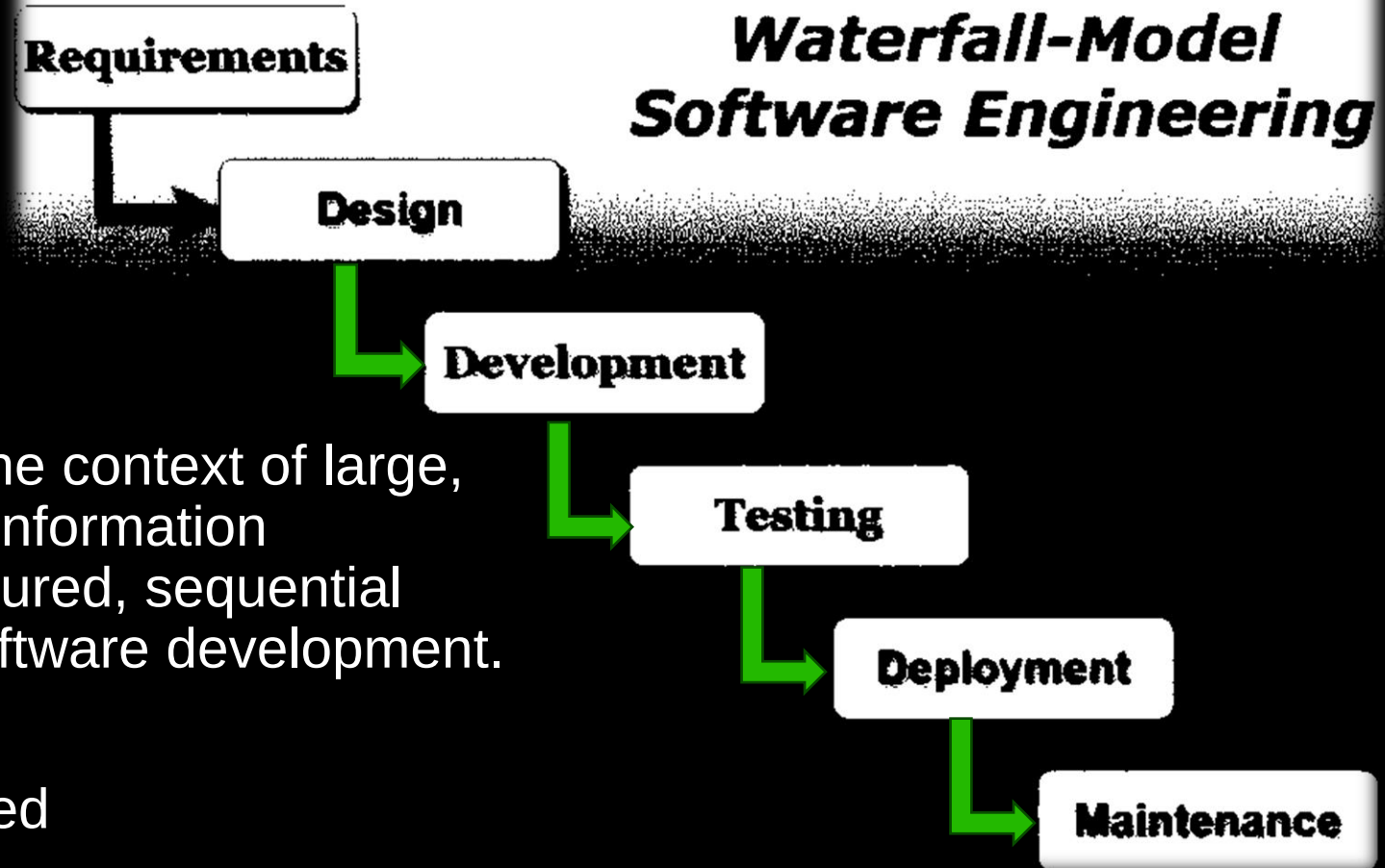
➔ SDLC specifies the task(s) to be performed at various stages. It ensures that the end product is able to meet the customer's expectations and fits within the budget. SDLC is a collection of these six stages,



SOFTWARE DEVELOPMENT LIFE CYCLE MODELS

- »» waterfall model
- »» Spiral Model
- »» V-Shaped Model
- »» Iterative Model
- »» RAD
- »» RSD-Agile model(XP, Scrum)

SDLC –WATERFALL



A software development model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to project management and software development.

Typically used when

- project requirements are well-defined
- project goals are clear
- large-scale projects with long timelines

SDLC –WATERFALL

1. Requirements

➤ Feasibility study

Involves understanding the problem and then determining the various possible strategies to solve the problem. These different identified solutions are analyzed based on their benefits and drawbacks. The best solution is chosen and all the other phases are carried out as per this solution strategy.

➤ Requirement analysis and specification.

This phase aims to understand the exact requirements of the customer and document them properly. This phase consists of two different activities.

- Requirement gathering and analysis: gathered requirements are analyzed. The goal of the analysis part is to remove incompleteness (when some parts of the actual requirements have been omitted) and inconsistencies (when some part of the requirement contradicts some other part).
- Requirement specification: These analyzed requirements are documented in a software requirement specification (SRS) document. SRS document serves as a contract between the development team and customers. Any future dispute between the customers and the developers can be settled by examining the SRS document.

SDLC –WATERFALL

2. Design:

- » In the design phase we convert the requirements acquired in the SRS into a format that can be coded in a programming language. It includes high-level(HLD) and detailed design(LLD) as well as the overall software architecture. A Software Design Document(SDD) is used to document all of this effort

3. Development:

- » In the development or coding phase the software design is translated into source code using any suitable programming language. Thus each designed module is coded.

Integration -The unit testing phase aims to check whether each module is working properly or not is done before integration

- » Integration of different modules is undertaken soon after they have been coded and unit tested. Integration of various modules is carried out incrementally over several steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested.

4. Testing

- » Finally, after all the modules have been successfully integrated and tested, the full working system is obtained and system testing is carried out on this. System testing consists of three different kinds of testing activities as described below.

- Alpha testing: Alpha testing is the system testing performed by the development team.
- Beta testing: Beta testing is the system testing performed by a friendly set of customers.
- Acceptance testing: After the software has been delivered, the customer performs acceptance testing to determine whether to accept the delivered software or reject it.

SDLC –WATERFALL

5. Deployment

- The software is deployed into a live environment (client's server) in order to test its performance.
- Quality assurance through a thorough testing to ensure the software meets all requirements and is free from defects. Once the software is deployed, it becomes available to end-users.
- At this point User acceptance testing (UAT) is done: Involving end-users in testing to validate that the software meets their needs and expectations. Sometimes, this phase also includes training of real-time users to communicate benefits of the system.

6. Maintenance and evolution:

- Maintenance is the most important phase of a software life cycle. The effort spent on maintenance is 60% of the total effort spent to develop a full software. There are three types of maintenance.
 - Corrective Maintenance: This type of maintenance is carried out to correct errors that were not discovered during the product development phase.
 - Perfective Maintenance: This type of maintenance is carried out to enhance the functionalities of the system based on the customer's request.
 - Adaptive Maintenance: Adaptive maintenance is usually required for porting the software to work in a new environment such as working on a new computer platform or with a new operating system.

SDLC –WATERFALL

➤ Advantages of this SDLC model.

- Easy to Understand: The Classical Waterfall Model is very simple and easy to understand.
- Individual Processing: Phases in the Classical Waterfall model are processed one at a time.
- Properly Defined: In the classical waterfall model, each stage in the model is clearly defined.
- Clear Milestones: The classical Waterfall model has very clear and well-understood milestones.
- Properly Documented: Processes, actions, and results are very well documented.
- Reinforces Good Habits: The Classical Waterfall Model reinforces good habits like define-before-design and design-before-code.
- Working: Classical Waterfall Model works well for smaller projects and projects where requirements are well understood.

SDLC –WATERFALL

- Drawbacks of the waterfall model.
 - No Feedback Path
 - Difficult to accommodate Change Requests
 - No Overlapping of Phases
 - Limited Flexibility
 - Limited Stakeholder Involvement
 - Late Defect Detection
 - Lengthy Development Cycle

SDLC –WATERFALL

➤ When to use the SDLC Waterfall Model

- Well-understood Requirements.
- Very Little Changes Expected.
- Small to Medium-Sized Projects.
- Predictable.
- Regulatory Compliance is Critical.
- Client Prefers a Linear and Sequential Approach
- Limited Resources.

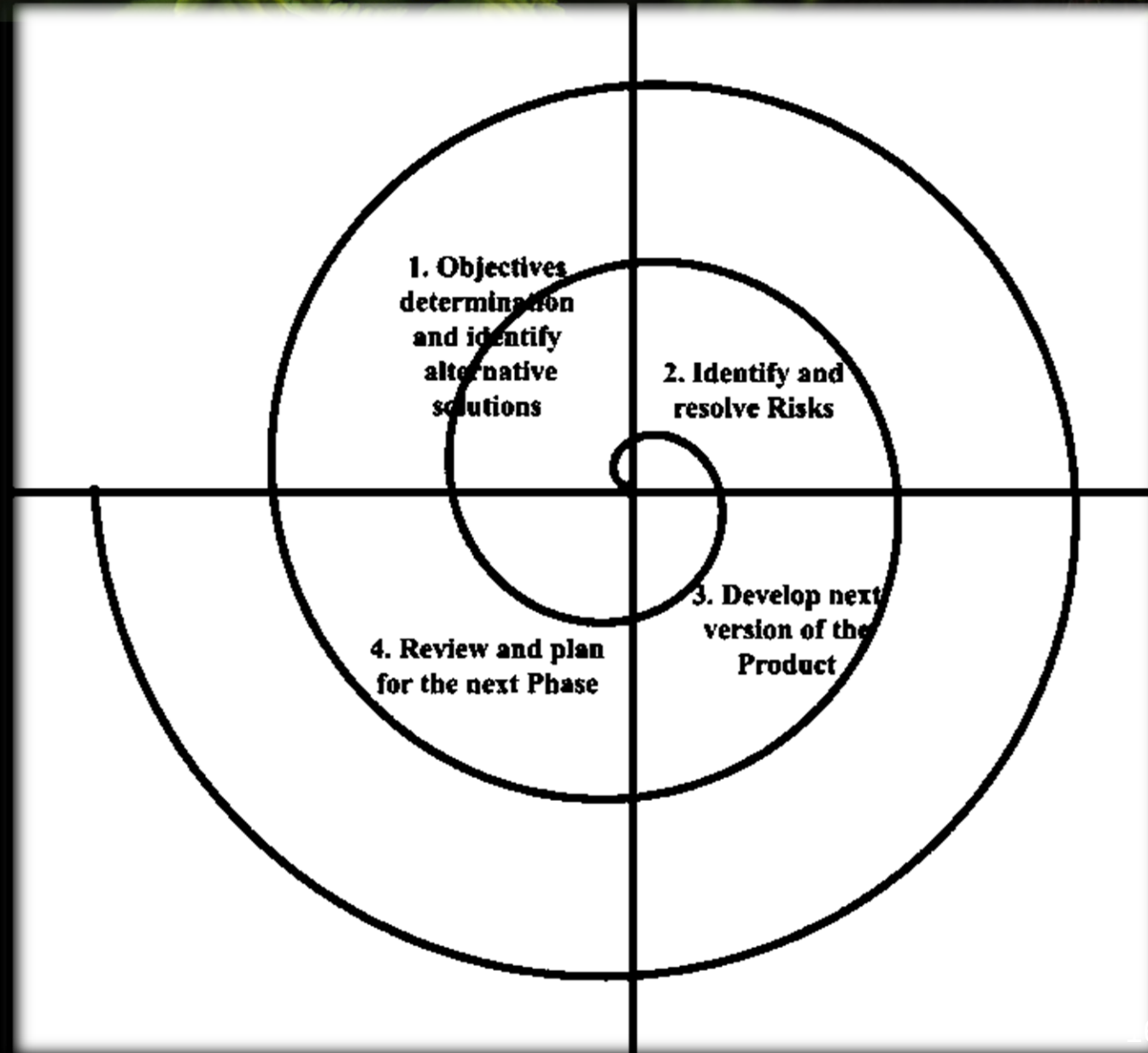
SDLC –WATERFALL

➤ Applications of SDLC Waterfall Model

- Large-scale Software Development Projects: The model is used for large-scale software development projects, where a structured and sequential approach is necessary to ensure that the project is completed on time and within budget.
- Safety-Critical Systems: Waterfall is ideal in the development of safety-critical systems, such as aerospace or medical systems, where the consequences of errors or defects can be severe.
- Government and Defense Projects: The Waterfall Model is also commonly used in government and defense projects, where a rigorous and structured approach is necessary to ensure that the project meets all requirements and is delivered on time.
- Projects with well-defined Requirements: The Model is best suited for projects with well-defined requirements, as the sequential nature of the model requires a clear understanding of the project objectives and scope.
- Projects with Stable Requirements: Waterfall Model is also well-suited for projects with stable requirements, as the linear nature of the model does not allow for changes to be made once a phase has been completed.

SDLC SPIRAL MODEL

- The Spiral Model is a Software Development model that provides a systematic and iterative approach to software development.
- The Spiral Model is a combination of the waterfall model and the iterative model. It provides support for Risk Handling. The Spiral Model was first proposed by Barry Boehm.



SDLC SPIRAL MODEL

Some Key Points regarding the phase of a Spiral Model:

- The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks.
- As the project manager dynamically determines the number of phases, the project manager has an important role in developing a product using the spiral model.
- It is based on the idea of a spiral, with each iteration of the spiral representing a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and maintenance.

SDLC SPIRAL MODEL



The phase of the Spiral Model is divided into four quadrants as represented in the model. The functions of these four quadrants are:

1. Objectives determination and identify alternative solutions: Requirements are gathered from the customers and the objectives are identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.
2. Identify and resolve Risks: During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.
3. Develop the next version of the Product: During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.
4. Review and plan for the next Phase: In the fourth quadrant, the Customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.

SDLC SPIRAL MODEL

➤ Advantages of the Spiral Model.

1. Risk Handling-the model accommodates risk analysis and risk handling at every phase.
2. Good for large projects-large and complex projects.
3. Flexibility in Requirements-Change requests in the Requirements at a later phase can be incorporated accurately by using this model.
4. Customer Satisfaction
5. Iterative and Incremental Approach-allowing for flexibility and adaptability in response to changing requirements or unexpected events.
6. Emphasis on Risk Management.
7. Improved Communication.
8. Improved Quality

SDLC SPIRAL MODEL

»» Drawbacks of the spiral model.

1. Complex: The Spiral Model is much more complex than other SDLC models.
2. Expensive: Spiral Model is not suitable for small projects as it is expensive.
3. Too much dependability on Risk Analysis: The successful completion of the project is very much dependent on Risk Analysis. Without very highly experienced experts, it is going to be a failure to develop a project using this model.
4. Difficulty in time management: As the number of phases is unknown at the start of the project, time estimation is very difficult.
5. Complexity: The Spiral Model can be complex, as it involves multiple iterations of the software development process.
6. Time-Consuming: The Spiral Model can be time-consuming, as it requires multiple evaluations and reviews.
7. Resource Intensive: The Spiral Model can be resource-intensive, as it requires a significant investment in planning, risk analysis, and evaluations.

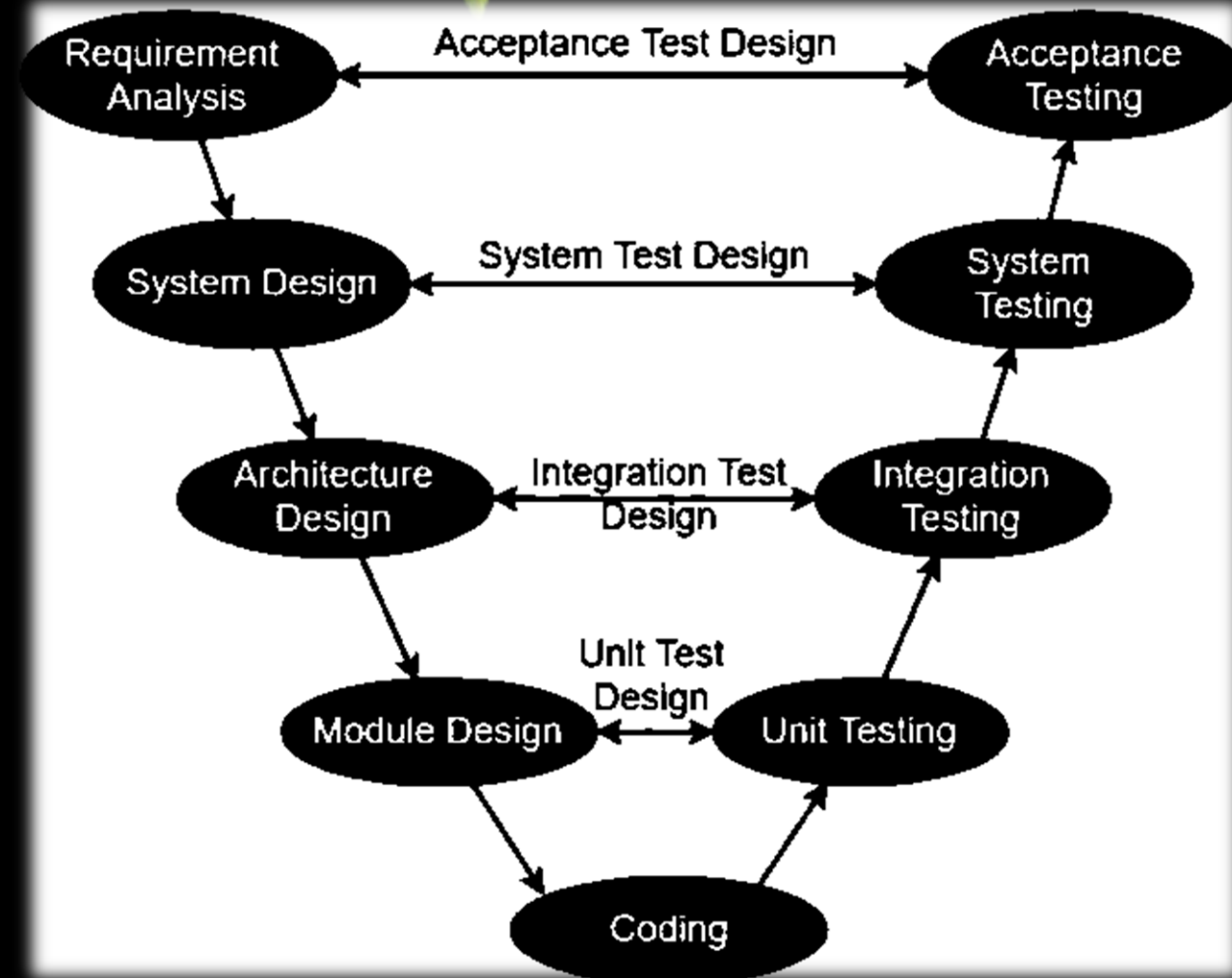
SDLC SPIRAL MODEL

»»When To Use the Spiral Model?

1. A spiral approach is utilized when frequent releases are necessary.
2. When it is appropriate to create a prototype
3. When evaluating risks and costs is crucial
4. The spiral approach is beneficial for projects with moderate to high risk.
5. The SDLC's spiral model is helpful when requirements are complicated and ambiguous.
6. If modifications are possible at any moment
7. When committing to a long-term project is impractical owing to shifting economic priorities.

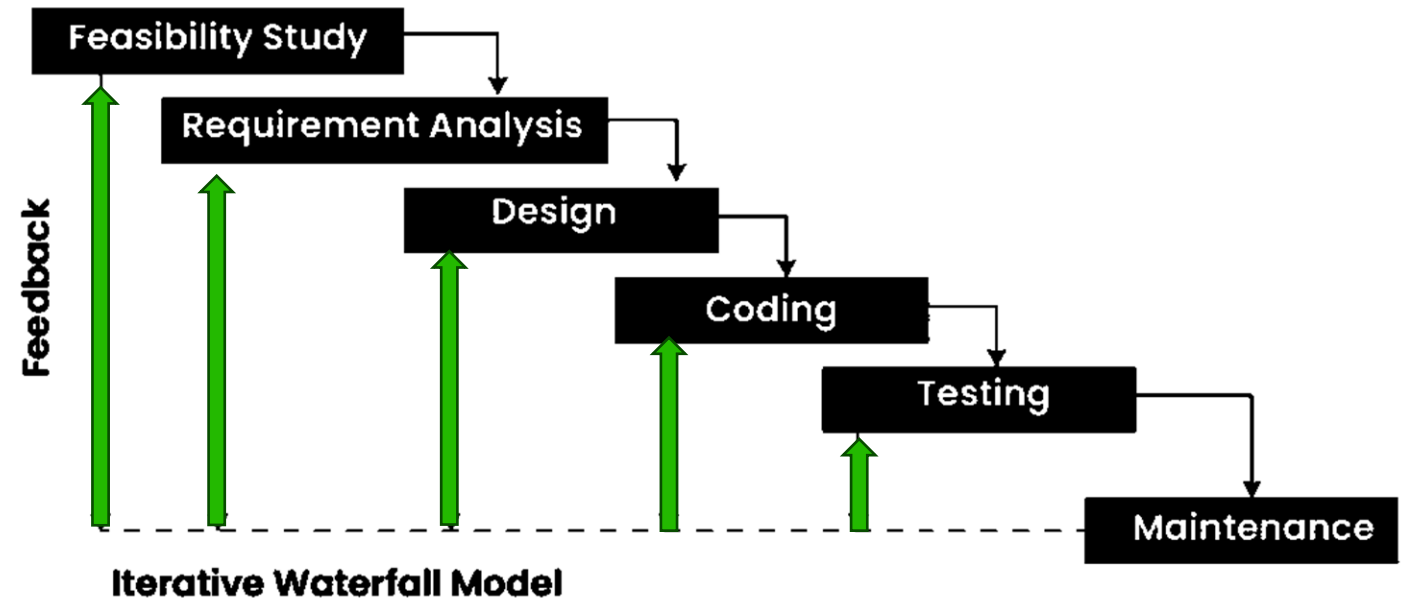
SDLC V-MODEL

- The V-model is a type of software development model where the process executes sequentially in a V-shape. It is also known as the Verification and Validation model. It is based on the association of a testing phase for each corresponding development stage.



SDLC ITERATIVE-MODEL

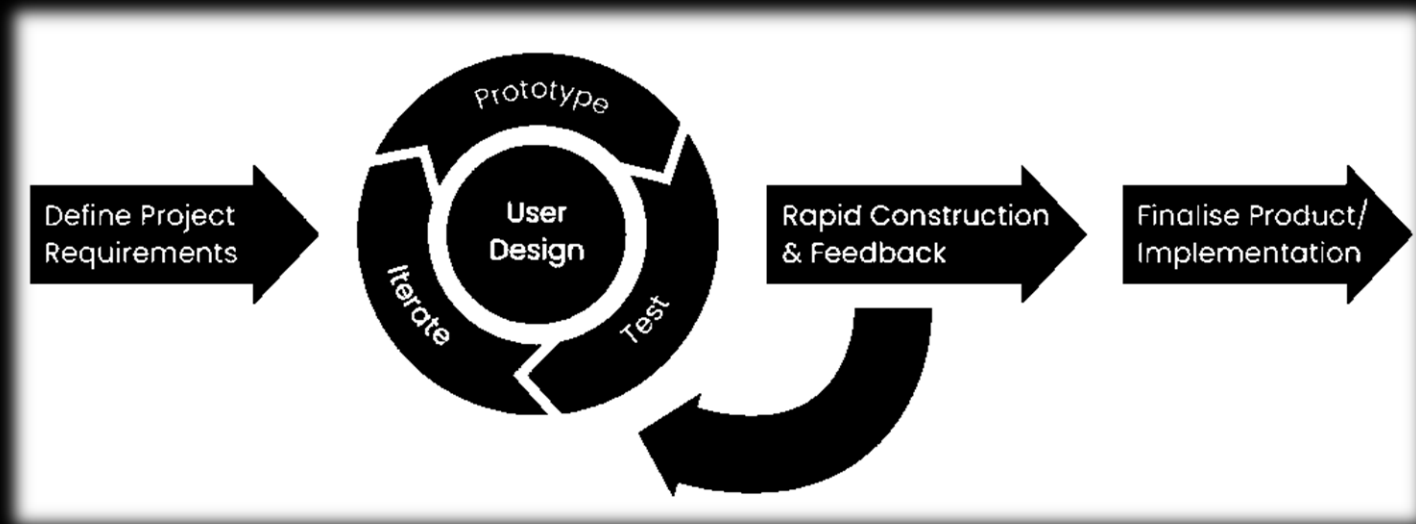
- The Iterative model combines the sequential steps of the traditional Waterfall Model with the flexibility of iterative design. It allows for improvements and changes to be made at each stage of the development process, instead of waiting until the end of the project.



Phases of Iterative Waterfall Model

SDLC RAPID APPLICATION DEVELOPMENT -MODEL

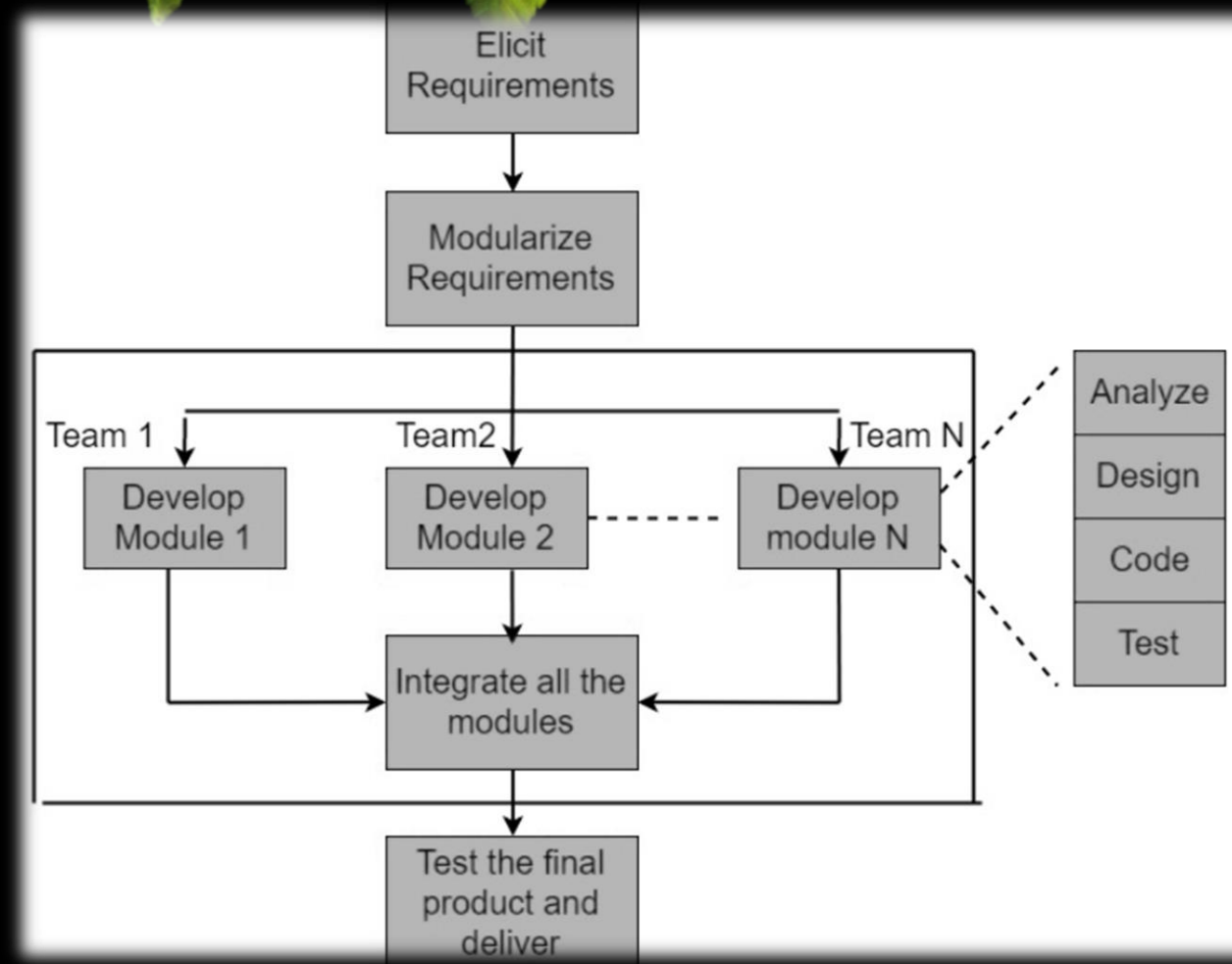
- The Model was first proposed by IBM in the 1980s. It is a type of incremental process model in which there is an extremely short development cycle. When the requirements are fully understood and the component-based construction approach is adopted then the RAD model is used. Various phases in RAD are Requirements Gathering, Analysis and Planning, Design, Build or Construction (prototyping), and finally Deployment.



SDLC RAPID APPLICATION DEVELOPMENT -MODEL

Basic phases:

1. Requirements Planning – This involves the use of various techniques used in requirements elicitation like brainstorming, task analysis, form analysis, user scenarios, FAST (Facilitated Application Development Technique), etc. It also consists of the entire structured plan describing the critical data, methods to obtain it, and then processing it to form a final refined model.
2. User Description – This phase consists of taking user feedback and building the prototype using developer tools. In other words, it includes re-examination and validation of the data collected in the first phase. The dataset attributes are also identified and elucidated in this phase.
3. Construction – In this phase, refinement of the prototype and delivery takes place. It includes the actual use of powerful automated tools to transform processes and data models into the final working product. All the required modifications and enhancements are to be done in this phase.
4. Cutover – All the interfaces between the independent modules developed by separate teams have to be tested properly. The use of powerfully automated tools and subparts makes testing easier. This is followed by acceptance testing by the user.



SDLC RSD-AGILE MODEL

RAPID SOFTWARE DEVELOPMENT

- Rapid development and delivery is now often the most important requirement for software systems
 - Businesses operate in a fast –changing requirement and it is practically impossible to produce a set of stable software requirements
 - Software has to evolve quickly to reflect changing business needs.
- Plan-driven development is essential for some types of system but does not meet these business needs.
- Agile development methods emerged in the late 1990s whose aim was to radically reduce the delivery time for working software systems

SDLC RSD-AGILE MODEL

AGILE DEVELOPMENT

- Program specification, design and implementation are interleaved
- The system is developed as a series of versions or increments with stakeholders involved in version specification and evaluation
- Frequent delivery of new versions for evaluation
- Extensive tool support (e.g. automated testing tools) used to support development.
- Minimal documentation – focus on working code

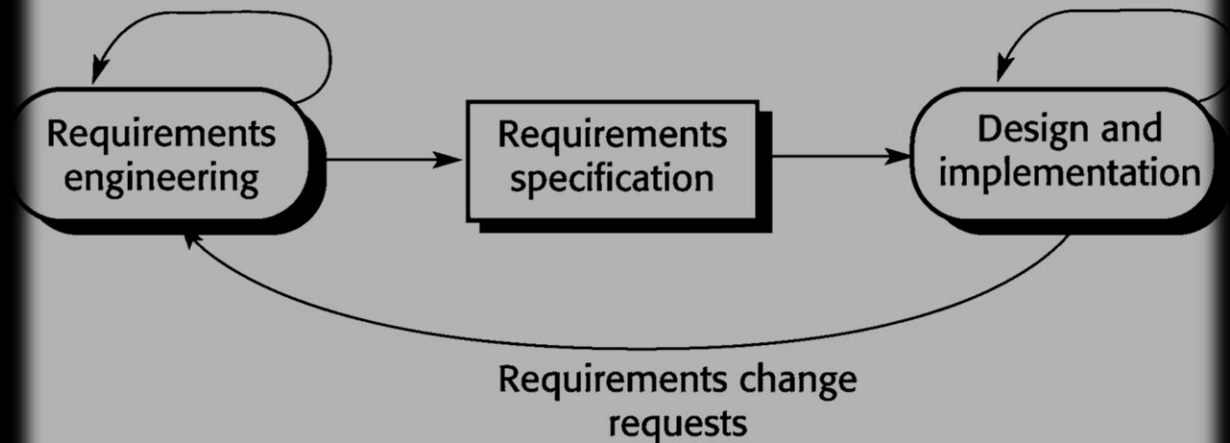
SDLC RSD-AGILE MODEL

PLAN-DRIVEN AND AGILE DEVELOPMENT

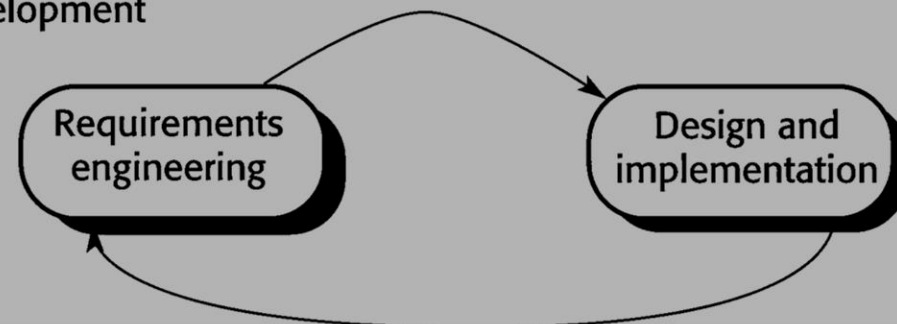
- Plan-driven development
 - Not necessarily waterfall model – plan-driven, incremental development is possible.

- Agile development
 - Specification, design, implementation and testing are interleaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Plan-based development



Agile development



SDLC RSD-AGILE MODEL

AGILE METHODS

- Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile methods. These methods:
 - Focus on the code rather than the design
 - Are based on an iterative approach to software development
 - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- The aim of agile methods is to reduce overheads in the software process and to be able to respond quickly to changing requirements without excessive rework.

SDLC RSD-AGILE MODEL

AGILE MANIFESTO

- We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:
 - Individuals and interactions over processes and tools
 - Working software over comprehensive documentation
 - Customer collaboration over contract negotiation
 - Responding to change over following a plan
- That is, while there is value in the items on the right, we value the items on the left more.

SDLC RSD-AGILE MODEL

THE PRINCIPLES OF AGILE METHODS

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

SDLC RSD-AGILE MODEL

AGILE METHOD APPLICABILITY

- Product development where a software company is developing a small or medium-sized product for sale.
 - Virtually all software products and apps are now developed using an agile approach
- Custom system development within an organization, where there is a clear commitment from the customer to become involved in the development process and where there are few external rules and regulations that affect the software.

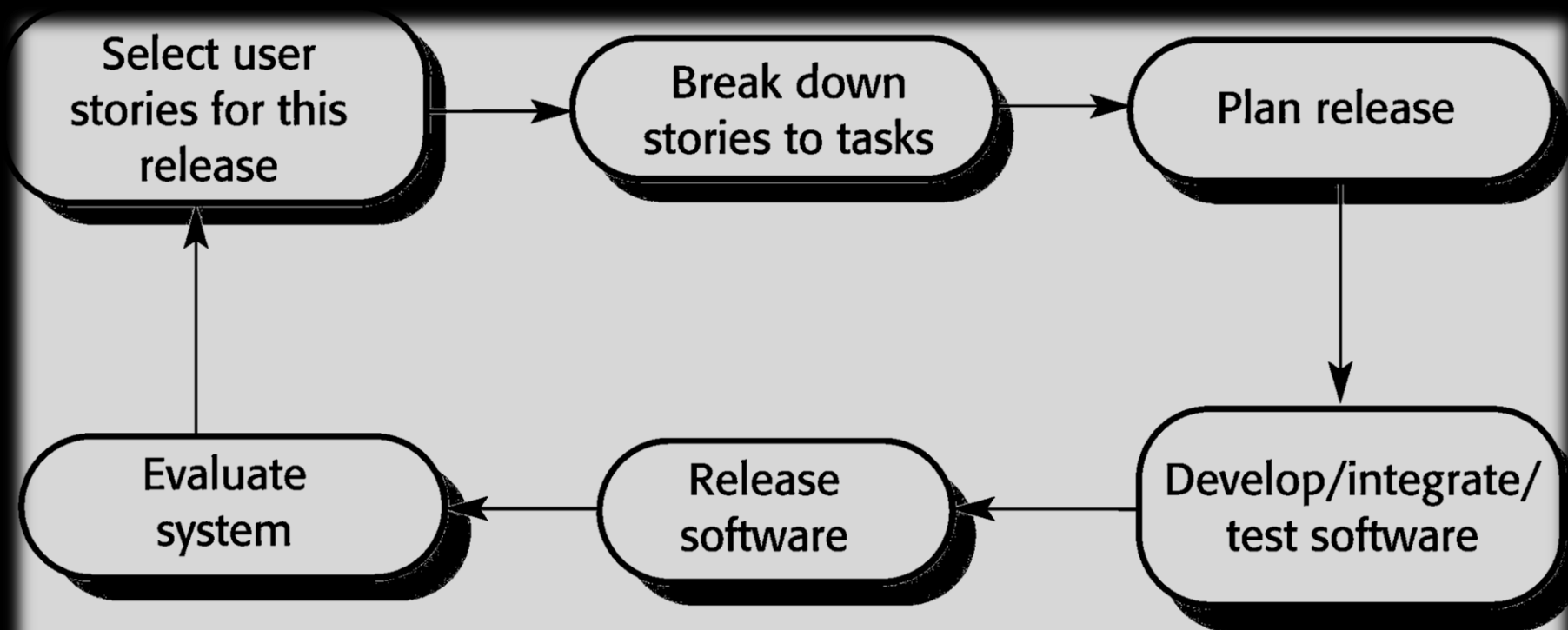
SDLC RSD-AGILE MODEL

EXTREME PROGRAMMING

- A very influential agile method, developed in the late 1990s, that introduced a range of agile development techniques.
- Extreme Programming (XP) takes an 'extreme' approach to iterative development.
 - New versions may be built several times per day;
 - Increments are delivered to customers every 2 weeks;
 - All tests must be run for every build and the build is only accepted if tests run successfully.

SDLC RSD-AGILE MODEL

THE EXTREME PROGRAMMING RELEASE CYCLE



Copyright ©2016 Pearson Education, All Rights Reserved

SDLC RSD-AGILE MODEL

EXTREME PROGRAMMING PRACTICES (A)

Principle / practice	Description
Incremental planning	Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development 'Tass'.
Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to meet the current requirements and no more.
Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

EXTREME PROGRAMMING PRACTICES (B)

Principle / practice		Description
Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.	
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.	
Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.	
Sustainable pace	Large amounts of overtime are not considered acceptable as the net effect is often to reduce code quality and medium term productivity	
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In XP process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.	

SDLC RSD-AGILE MODEL

XP AND AGILE PRINCIPLES

- Incremental development is supported through small, frequent system releases.
- Customer involvement means full-time customer engagement with the team.
- People not process through pair programming, collective ownership and a process that avoids long working hours.
- Change supported through regular system releases.
- Maintaining simplicity through constant refactoring of code.

SDLC RSD-AGILE MODEL

INFLUENTIAL XP PRACTICES

- Extreme programming has a technical focus and is not easy to integrate with management practice in most organizations.
- Consequently, while agile development uses practices from XP, the method as originally defined is not widely used.
- Key practices
 - User stories for specification
 - Refactoring
 - Test-first development
 - Pair programming

SDLC RSD-AGILE MODEL

USER STORIES FOR REQUIREMENTS

- In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- User requirements are expressed as user stories or scenarios.
- These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

SDLC RSD-AGILE MODEL

REFACTORING (1)

- Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated.
- Rather, it proposes constant code improvement (refactoring) to make changes easier when they have to be implemented.

SDLC RSD-AGILE MODEL

REFACTORING (2)

- Programming team look for possible software improvements and make these improvements even where there is no immediate need for them.
- This improves the understandability of the software and so reduces the need for documentation.
- Changes are easier to make because the code is well-structured and clear.
- However, some changes requires architecture refactoring and this is much more expensive.

SDLC RSD-AGILE MODEL

TEST-FIRST DEVELOPMENT

- Testing is central to XP and XP has developed an approach where the program is tested after every change has been made.
- XP testing features:
 - Test-first development.
 - Incremental test development from scenarios.
 - User involvement in test development and validation.
 - Automated test harnesses are used to run all component tests each time that a new release is built.

SDLC RSD-AGILE MODEL

TEST-DRIVEN DEVELOPMENT

- Writing tests before code clarifies the requirements to be implemented.
- Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
 - Usually relies on a testing framework such as Junit.
- All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors.

SDLC RSD-AGILE MODEL

CUSTOMER INVOLVEMENT

- The role of the customer in the testing process is to help develop acceptance tests for the stories that are to be implemented in the next release of the system.
- The customer who is part of the team writes tests as development proceeds. All new code is therefore validated to ensure that it is what the customer needs.
- However, people adopting the customer role have limited time available and so cannot work full-time with the development team. They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process.

SDLC RSD-AGILE MODEL

TEST AUTOMATION

- »» Test automation means that tests are written as executable components before the task is implemented
 - These testing components should be stand-alone, should simulate the submission of input to be tested and should check that the result meets the output specification. An automated test framework is a system that makes it easy to write executable tests and submit a set of tests for execution.
- »» As testing is automated, there is always a set of tests that can be quickly and easily executed
 - Whenever any functionality is added to the system, the tests can be run and problems that the new code has introduced can be caught immediately.

SDLC RSD-AGILE MODEL

PROBLEMS WITH TEST-FIRST DEVELOPMENT

- Programmers prefer programming to testing and sometimes they take short cuts when writing tests. For example, they may write incomplete tests that do not check for all possible exceptions that may occur.
- Some tests can be very difficult to write incrementally. For example, in a complex user interface, it is often difficult to write unit tests for the code that implements the 'display logic' and workflow between screens.
- It difficult to judge the completeness of a set of tests. Although you may have a lot of system tests, your test set may not provide complete coverage.

SDLC RSD-AGILE MODEL

PAIR PROGRAMMING (1)

- Pair programming involves programmers working in pairs, developing code together.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from improving the system code.

SDLC RSD-AGILE MODEL

PAIR PROGRAMMING (2)

- In pair programming, programmers sit together at the same computer to develop the software.
- Pairs are created dynamically so that all team members work with each other during the development process.
- The sharing of knowledge that happens during pair programming is very important as it reduces the overall risks to a project when team members leave.
- Pair programming is not necessarily inefficient and there is some evidence that suggests that a pair working together is more efficient than 2 programmers working separately.

SDLC RSD-AGILE MODEL

AGILE PROJECT MANAGEMENT

- »» The principal responsibility of software project managers is to manage the project so that the software is delivered on time and within the planned budget for the project.
- »» The standard approach to project management is plan-driven. Managers draw up a plan for the project showing what should be delivered, when it should be delivered and who will work on the development of the project deliverables.
- »» Agile project management requires a different approach, which is adapted to incremental development and the practices used in agile methods.

SDLC RSD-AGILE MODEL

SCRUM

- »» Scrum is an agile method that focuses on managing iterative development rather than specific agile practices.
- »» There are three phases in Scrum.
 - The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
 - This is followed by a series of sprint cycles, where each cycle develops an increment of the system.
 - The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

SDLC RSD-AGILE MODEL

SCRUM TERMINOLOGY (A)

Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than 7 people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be 'potentially shippable' which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of 'to do' items which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously review the product backlog to ensure that the project continues to meet critical business needs.

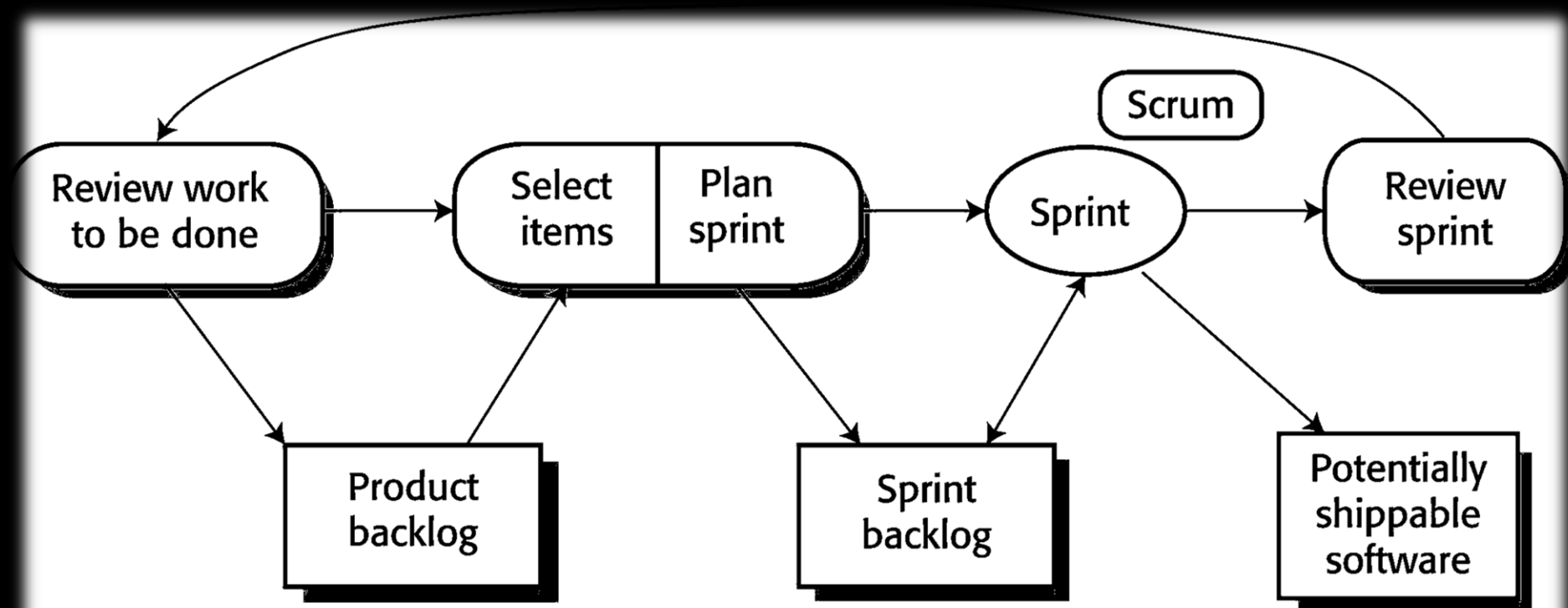
SDLC RSD-AGILE MODEL

SCRUM TERMINOLOGY (B)

Scrum term	Definition
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager.
Sprint	A development iteration. Sprints are usually 2-4 weeks long.
Velocity	An estimate of how much product backlog effort that a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

SDLC RSD-AGILE MODEL

SCRUM SPRINT CYCLE



SDLC RSD-AGILE MODEL

TEAMWORK IN SCRUM

- The 'Scrum master' is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- The whole team attends short daily meetings (Scrums) where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.

SDLC RSD-AGILE MODEL

SCRUM BENEFITS

- The product is broken down into a set of manageable and understandable chunks.
- Unstable requirements do not hold up progress.
- The whole team have visibility of everything and consequently team communication is improved.
- Customers see on-time delivery of increments and gain feedback on how the product works.
- Trust between customers and developers is established and a positive culture is created in which everyone expects the project to succeed.

SDLC RSD-AGILE MODEL

SCALING AGILE METHODS

- Agile methods have proved to be successful for small and medium sized projects that can be developed by a small co-located team.
- It is sometimes argued that the success of these methods comes because of improved communications which is possible when everyone is working together.
- Scaling up agile methods involves changing these to cope with larger, longer projects where there are multiple development teams, perhaps working in different locations.

SDLC RSD-AGILE MODEL

SCALING OUT AND SCALING UP

- 'Scaling up' is concerned with using agile methods for developing large software systems that cannot be developed by a small team.
- 'Scaling out' is concerned with how agile methods can be introduced across a large organization with many years of software development experience.
- When scaling agile methods it is important to maintain agile fundamentals:
 - Flexible planning, frequent system releases, continuous integration, test-driven development and good team communications.

SDLC RSD-AGILE MODEL

PRACTICAL PROBLEMS WITH AGILE METHODS

- The informality of agile development is incompatible with the legal approach to contract definition that is commonly used in large companies.
- Agile methods are most appropriate for new software development rather than software maintenance. Yet most software costs in large companies come from maintaining their existing software systems.
- Agile methods are designed for small co-located teams yet much software development now involves worldwide distributed teams.

SDLC RSD-AGILE MODEL

PROCESS MEASUREMENT

- Most software contracts for custom systems are based around a specification, which sets out what has to be implemented by the system developer for the system customer.
- However, this precludes interleaving specification and development as is the norm in agile development.
- A contract that pays for developer time rather than functionality is required.
 - However, this is seen as a high risk by many legal departments because what has to be delivered cannot be guaranteed.

SDLC RSD-AGILE MODEL

AGILE METHODS AND SOFTWARE MAINTENANCE

- Most organizations spend more on maintaining existing software than they do on new software development. So, if agile methods are to be successful, they have to support maintenance as well as original development.
- Two key issues:
 - Are systems that are developed using an agile approach maintainable, given the emphasis in the development process of minimizing formal documentation?
 - Can agile methods be used effectively for evolving a system in response to customer change requests?
- Problems may arise if original development team cannot be maintained.

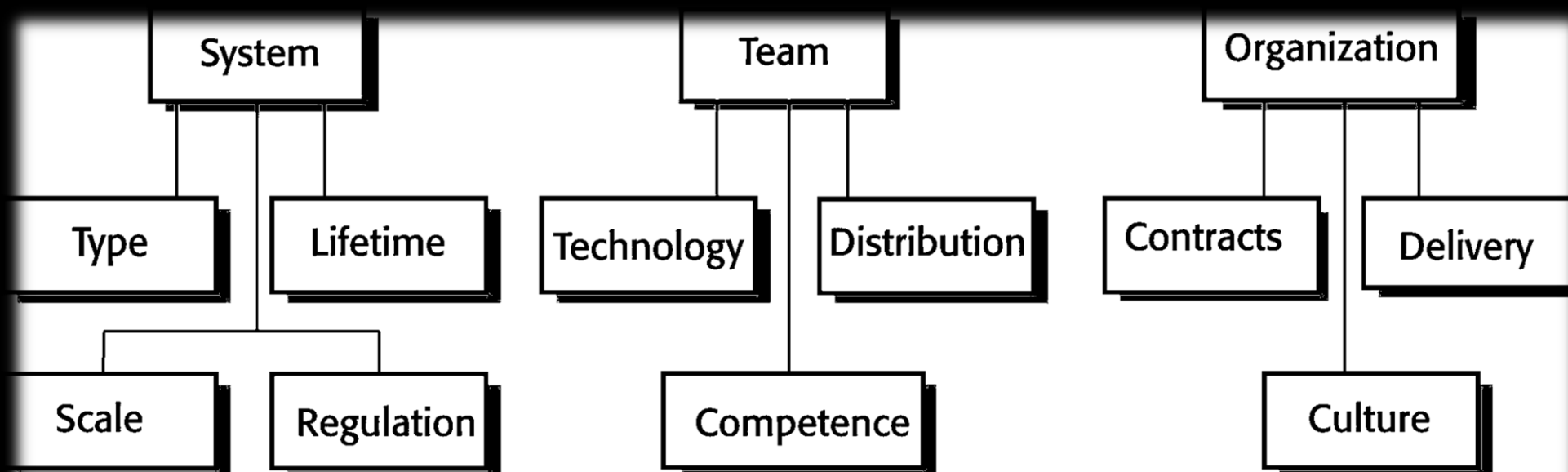
SDLC RSD-AGILE MODEL

AGILE MAINTENANCE

- Key problems are:
 - Lack of product documentation
 - Keeping customers involved in the development process
 - Maintaining the continuity of the development team
- Agile development relies on the development team knowing and understanding what has to be done.
- For long-lifetime systems, this is a real problem as the original developers will not always work on the system.

SDLC RSD-AGILE MODEL

AGILE AND PLAN-BASED FACTORS



Copyright ©2016 Pearson Education, All Rights Reserved

SDLC RSD-AGILE MODEL

SYSTEM ISSUES

- How large is the system being developed?
 - Agile methods are most effective a relatively small co-located team who can communicate informally.
- What type of system is being developed?
 - Systems that require a lot of analysis before implementation need a detailed design to carry out this analysis.
- What is the expected system lifetime?
 - Long-lifetime systems require documentation to communicate the intentions of the system developers to the support team.
- Is the system subject to external regulation?
 - If a system is regulated you will probably be required to produce detailed documentation as part of the system safety case.

SDLC RSD-AGILE MODEL

PEOPLE AND TEAMS

- How good are the designers and programmers in the development team?
 - It is sometimes argued that agile methods require higher skill levels than plan-based approaches in which programmers simply translate a detailed design into code.
- How is the development team organized?
 - Design documents may be required if the team is distributed.
- What support technologies are available?
 - IDE support for visualisation and program analysis is essential if design documentation is not available.

SDLC RSD-AGILE MODEL

ORGANIZATIONAL ISSUES

- Traditional engineering organizations have a culture of plan-based development, as this is the norm in engineering.
- Is it standard organizational practice to develop a detailed system specification?
- Will customer representatives be available to provide feedback of system increments?
- Can informal agile development fit into the organizational culture of detailed documentation?

SDLC RSD-AGILE MODEL

AGILE METHODS FOR LARGE SYSTEMS

- Large systems are usually collections of separate, communicating systems, where separate teams develop each system. Frequently, these teams are working in different places, sometimes in different time zones.
- Large systems are 'brownfield systems', that is they include and interact with a number of existing systems. Many of the system requirements are concerned with this interaction and so don't really lend themselves to flexibility and incremental development.
- Where several systems are integrated to create a system, a significant fraction of the development is concerned with system configuration rather than original code development.

SDLC RSD-AGILE MODEL

LARGE SYSTEM DEVELOPMENT

- Large systems and their development processes are often constrained by external rules and regulations limiting the way that they can be developed.
- Large systems have a long procurement and development time. It is difficult to maintain coherent teams who know about the system over that period as, inevitably, people move on to other jobs and projects.
- Large systems usually have a diverse set of stakeholders. It is practically impossible to involve all of these different stakeholders in the development process.

SDLC RSD-AGILE MODEL

MULTI-TEAM SCRUM

➤ Role replication

- Each team has a Product Owner for their work component and ScrumMaster.

➤ Product architects

- Each team chooses a product architect and these architects collaborate to design and evolve the overall system architecture.

➤ Release alignment

- The dates of product releases from each team are aligned so that a demonstrable and complete system is produced.

➤ Scrum of Scrums

- There is a daily Scrum of Scrums where representatives from each team meet to discuss progress and plan work to be done.

SDLC RSD-AGILE MODEL

AGILE METHODS ACROSS ORGANIZATIONS

- Project managers who do not have experience of agile methods may be reluctant to accept the risk of a new approach.
- Large organizations often have quality procedures and standards that all projects are expected to follow and, because of their bureaucratic nature, these are likely to be incompatible with agile methods.
- Agile methods seem to work best when team members have a relatively high skill level. However, within large organizations, there are likely to be a wide range of skills and abilities.
- There may be cultural resistance to agile methods, especially in those organizations that have a long history of using conventional systems engineering processes.

SDLC - HOW TO CHOOSE AN SDLC MODEL?

Key factors to consider while choosing the right SDLC model.

■ Correct choice of SDLC model is essential for project success.

➤ Project Requirements:

1. Clear Requirements: Use Waterfall or V-Model if requirements are well-defined and unlikely to change.
2. Changing Requirements: Use Agile or Iterative models if requirements are unclear or likely to evolve.

➤ Project Size and Complexity:

2. Small Projects: Use Waterfall or RAD for small, simple projects.
3. Large Projects: Use Agile, Spiral, or DevOps for large, complex projects that need flexibility.

➤ Risk Management:

1. High-Risk Projects: Use Spiral for projects with significant risks and uncertainties.
2. Low-Risk Projects: Use Waterfall for projects with minimal risks.

SDLC - HOW TO CHOOSE AN SDLC MODEL?

➤ Team Expertise:

1. Experienced Teams: Use Agile or Scrum if the team is familiar with iterative development.
2. Less Experienced Teams: Use Waterfall or V-Model for teams needing structured guidance.

➤ Client Involvement:

1. Frequent Client Feedback: Use Agile, Scrum, or RAD if regular client interaction is needed.
2. Minimal Client Involvement: Use Waterfall or V-Model if client involvement is low after initial planning.

➤ Time and Budget Constraints:

1. Fixed Time and Budget: Use Waterfall or V-Model if you have strict time and budget limits.
2. Flexible Time and Budget: Use Agile or Spiral if you can adjust time and budget as needed.

SDLC - HOW TO CHOOSE AN SDLC MODEL?

➤ Product Release Timeline:

1. Quick Release Needed: Use Agile or RAD to deliver products quickly.
2. Longer Development Time: Use Waterfall or V-Model for projects with no urgent deadlines.

➤ Maintenance and Support:

1. Long-Term Maintenance: Use Agile or DevOps for projects needing continuous updates and support.
2. Minimal Maintenance: Use Waterfall or V-Model if little future maintenance is expected.

➤ Stakeholder Expectations:

1. High Stakeholder Engagement: Use Agile or Scrum if stakeholders want ongoing involvement.
2. Low Stakeholder Engagement: Use Waterfall or V-Model if stakeholders prefer involvement only at major milestones.



KEY TAKEAWAY

The SDLC in software engineering is an important framework for a better and more structured development of optimized software. In a world full of rapid evolution in technology, SDLC phases plays a crucial role in enabling some good and innovative solutions for helping users and organizations. Also, it's better to adapt SDLC principles to achieve software development goals effectively.

No software model is the best it depends of varied situations.



NEXT LECTURE

Software process models
Process activities
Software and change
Software process improvement

